PHASE 3 FINAL REPORT

Cryptocurrency Market Analytics and Investment Decision Support

Final Architecture, Data Model, Spark Analysis and FinOps Report

Team	Khushbakht (Member 1), Waniya (Member 2)
Platform	Databricks Free Edition (serverless), PySpark, Delta Lake, Unity Catalog, Power BI
Data source	CoinGecko market chart data, hourly, 94 cryptocurrencies, 4 Sep to 4 Oct 2026
Repository	`github.com/Waniya317/Cryptocurrency-Market-Analytics-and-Investment-Decision-Support`
Date	26 October 2026

1	Project summary
2	Architecture
3	Gold data model
4	Decision log
5	Spark execution and DAG analysis
6	Power BI dashboard design
7	Validation and data quality
8	Cost and FinOps
9	Limitations and future work

1 Project summary

This project ingests hourly market data for 94 cryptocurrencies, cleans and models it in a Databricks Lakehouse (Bronze, Silver, Gold) and serves a Power BI dashboard that supports investment analysis. The system reports and compares; it does not make automatic investment decisions.

- **Phase 1:** data collection scripts (`get_full_load.py`, `get_incremental_load.py`) calling the CoinGecko `market_chart/range` endpoint.
- **Phase 2:** Bronze and Silver layers with explicit schemas, idempotent MERGE loads, parameterised backfill, schema-drift quarantine and an audit log table.
- **Phase 3:** Gold star schema, automated refresh through an orchestrator notebook, BI views, Power BI dashboard and this report.

The CoinGecko API is blocked on Databricks Free Edition, so the collection scripts were run beforehand and the resulting JSON files were uploaded to a Unity Catalog Volume. The course instructor approved this manual ingestion.

Source data

File	Load type	Records	Coverage (UTC)
`crypto_hourly_full_load_100coins.json`	Full	66,401	2026-09-04 23:00 to 2026-10-04 09:00
`crypto_hourly_incremental_load_94coins.json`	Incremental	14,474	2026-09-28 00:00 to 2026-10-04 09:00

Each record holds `coin_id`, `timestamp` (milliseconds), `price`, `market_cap` and `total_volume`. The incremental window lies inside the full-load window, so it adds no new keys; this is used as a deliberate idempotency test (section 7).

Business questions

#	Question	Answered in
1	Performance of each coin over time	fact_coin_daily
2	Market capitalisation versus trading activity	fact_coin_daily, fact_market_hourly
3	Volatility	fact_coin_daily
4	Differences by day of week	fact_coin_daily, dim_date
5	Differences by hour of day	fact_market_hourly, dim_hour
6	Volume changes versus price movement	fact_market_hourly
7	Recent intraday behaviour	fact_market_hourly (last 24 h)
8	Recent versus historical behaviour	fact_market_hourly, vw_intraday_analysis
9, 10	Predicted direction and model accuracy (optional)	Not implemented (section 9)
11	Allocation scenarios under market conditions	fact_portfolio_daily, dim_scenario

2 Architecture

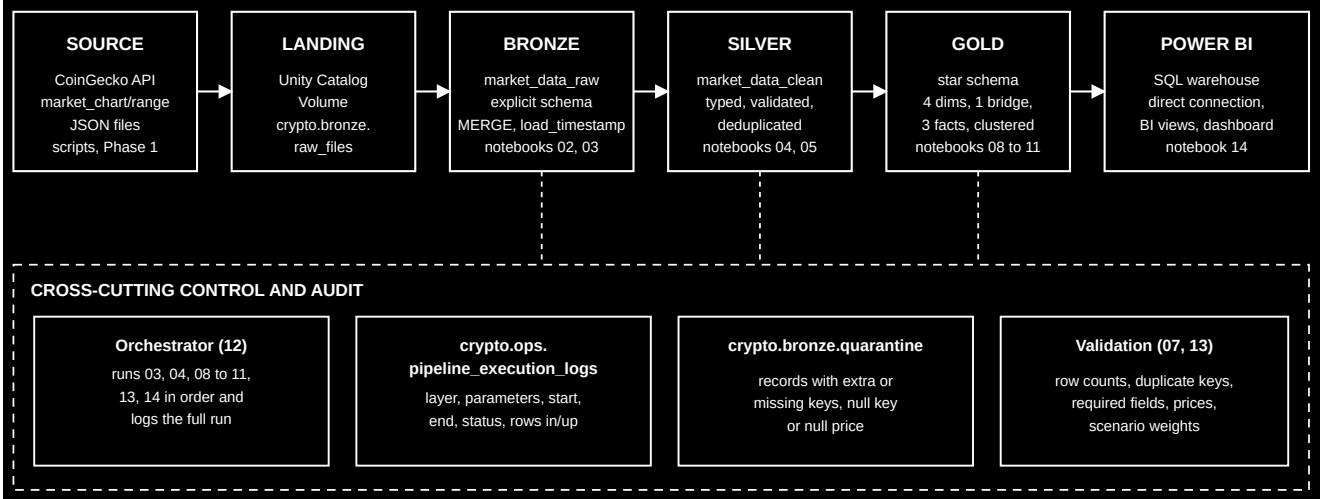


Figure 1. End-to-end data flow from the raw source to Power BI, with the control and audit components that run beside every layer.

Layer	Table	Rows	Key and purpose
Bronze	`crypto.bronze.market_data_raw`	66,401	(coin_id, ts_ms). Raw values, source_file, load_type, batch_id, load_timestamp.
Quarantine	`crypto.bronze.quarantine`	0	Rejected records with reason. The real source files contained no non-conforming records.
Silver	`crypto.silver.market_data_clean`	66,401	(coin_id, event_ts). Timestamp cast from milliseconds, event_date and event_hour derived.
Gold	`crypto.gold.*`	8 tables	Star schema, section 3.
Audit	`crypto.ops.pipeline_execution_logs`	1 row per run	run_id, layer, file_or_params, start_time, end_time, status, rows_inserted, rows_updated, error_message.

Phase 2 engineering controls

- **Explicit schemas:** all tables use `StructType` and `StructField` (notebook 00). Spark schema inference is not used.
- **Idempotency:** every write is a `MERGE INTO` on the business key that updates a row only when a value changed. Re-running a load reports 0 inserted, 0 updated.
- **Parameterised loads:** notebooks 02 and 03 take `path` and `batch_id` widgets; notebook 05 takes `start_date` and `end_date` for a Bronze to Silver backfill.
- **Audit:** every load logs layer, parameters, start and end time, status and rows inserted or updated through `log_run()`; failures are logged with the error text before the exception is re-raised.

3 Gold data model

The Gold layer is a star schema in `crypto.gold`: four dimensions, one bridge table and three fact tables. Surrogate keys are deterministic hashes (`coin_key = xxhash64(coin_id)`, `scenario_key =`

xxhash64(scenario_name)), date_key is an integer yyyymmdd and hour_key is 0 to 23.

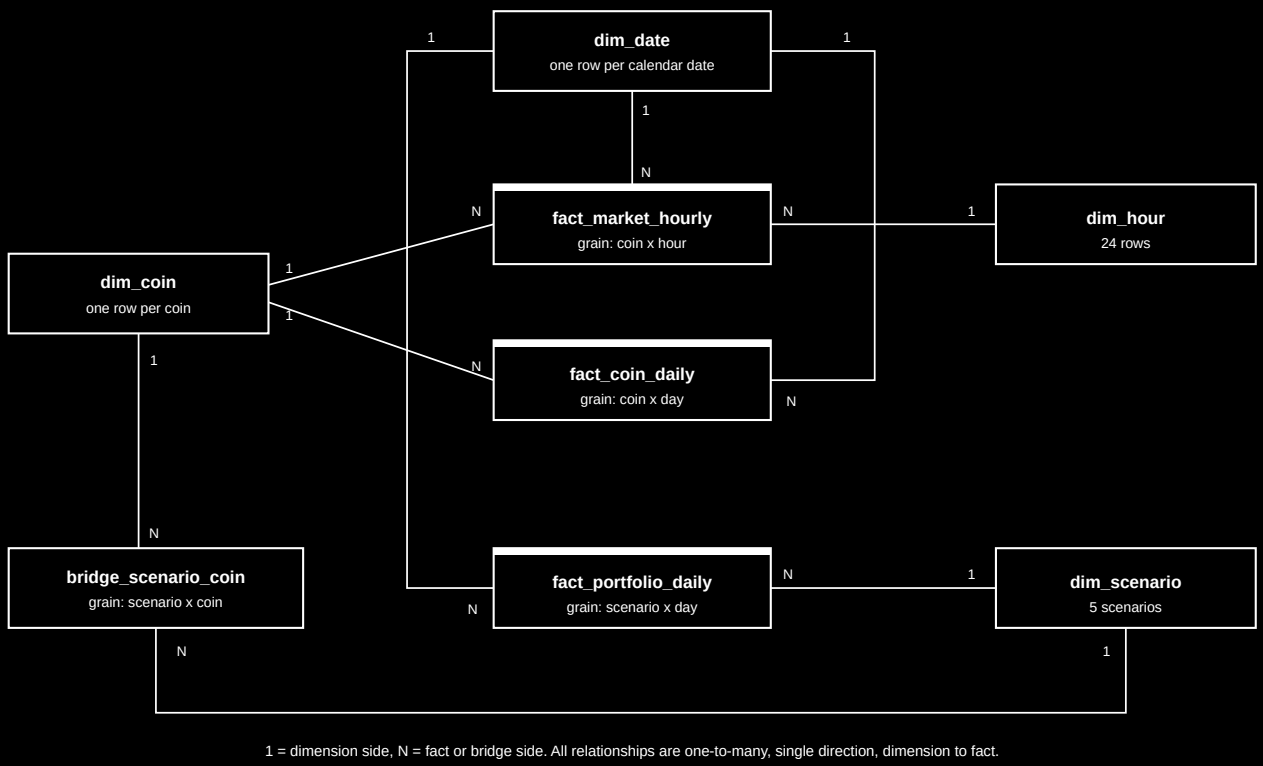


Figure 2. Entity-relationship diagram of the Gold star schema.

Table	Grain	Rows	Main columns
dim_coin	coin	94	coin_key, coin_id, coin_name, is_stablecoin, first_seen_ts, last_seen_ts
dim_date	calendar date	31	date_key, full_date, year, quarter, month, day_name, day_of_week, is_weekend
dim_hour	hour of day	24	hour_key, hour_label, day_part
dim_scenario	allocation scenario	5	scenario_key, scenario_name, description
bridge_scenario_coin	scenario x coin	282	scenario_key, coin_key, weight (sums to 1 per scenario)
fact_market_hourly	coin x hour	66,401	price, market_cap, total_volume, prev_price, hourly_return_pct, volume_change_pct, is_volume_spike, volume_to_mcap_ratio
fact_coin_daily	coin x day	2,914	open, close, high, low, avg price, avg_market_cap, avg_volume, daily_return_pct, intraday_range_pct, volatility_pct, hours_count
fact_portfolio_daily	scenario x day	155	portfolio_return_pct, market_condition (Bull, Bear, Sideways), is_high_volatility

Every Gold table carries load_timestamp. Informational PRIMARY KEY and FOREIGN KEY constraints are declared in Unity Catalog; Delta does not enforce them, but Power BI uses them to detect relationships. The five scenarios are Equal weight, BTC 60 / ETH 40, Top 5 by market cap, Stablecoin heavy and Altcoin heavy.

Refresh, clustering and OPTIMIZE

Gold tables are rebuilt from Silver and written with `MERGE INTO`, updating a row only when a measure changed. Floating-point measures are compared with a relative tolerance of 1e-9 so that recomputation noise in the last digits does not trigger false updates. The scenario and bridge tables also use `WHEN NOT MATCHED BY SOURCE THEN DELETE`, so stale membership rows disappear if a scenario definition changes. Each Gold step logs the rows it aggregated and its execution time to `pipeline_execution_logs`.

The fact tables use liquid clustering: `fact_market_hourly` and `fact_coin_daily` on (coin_key, date_key), `fact_portfolio_daily` on (date_key). `OPTIMIZE` compacts the files afterwards.

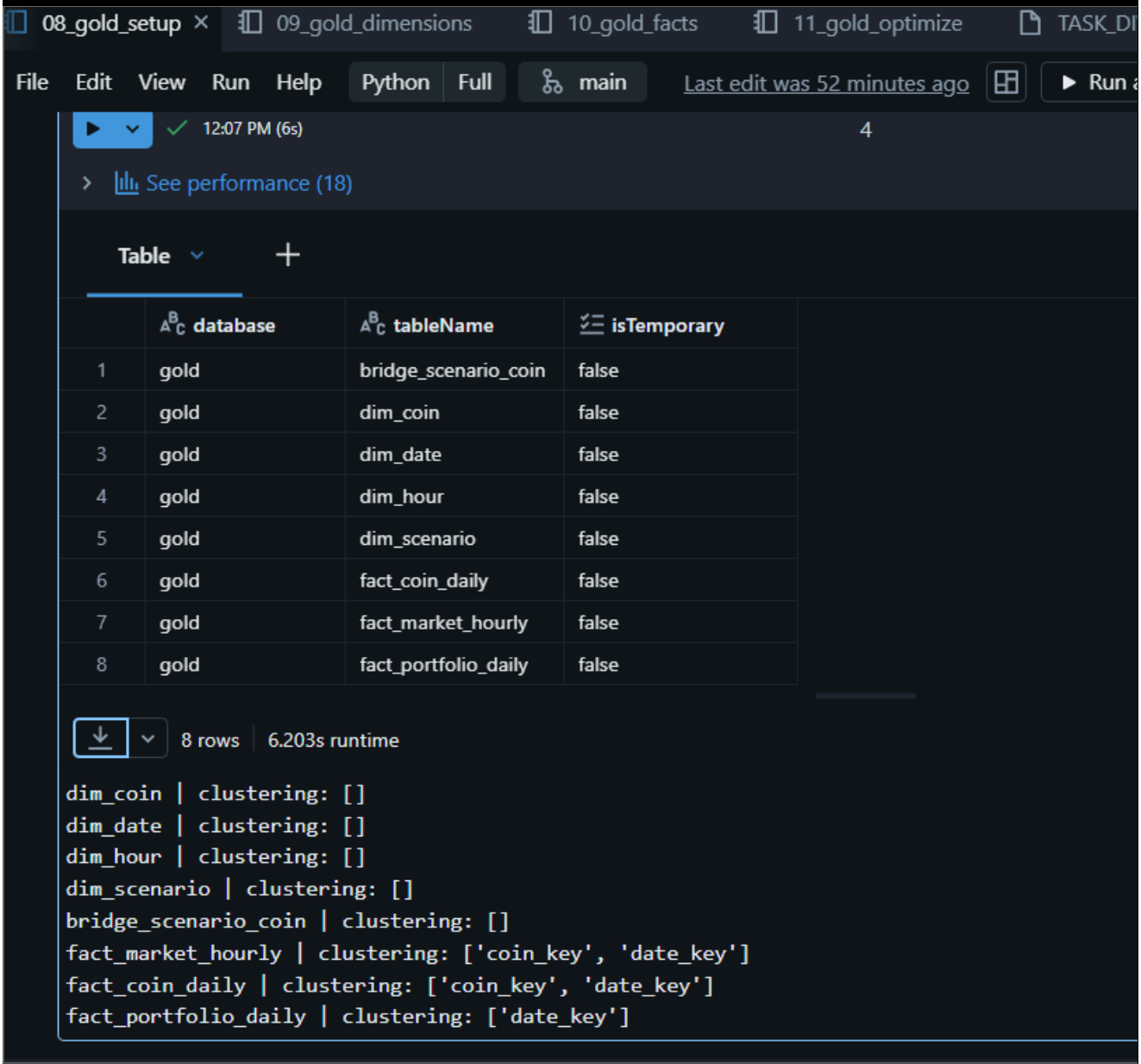


Figure 3. The eight Gold tables in `crypto.gold` and the clustering columns read back from the table metadata.

Table	Clustering	Files before	Files after	Bytes before	Bytes after
`fact_market_hourly`	`coin_key, date_key`	2	1	3,537,209	3,525,131
`fact_coin_daily`	`coin_key, date_key`	1	1	229,985	229,985
`fact_portfolio_daily`	`date_key`	1	1	3,726	3,726

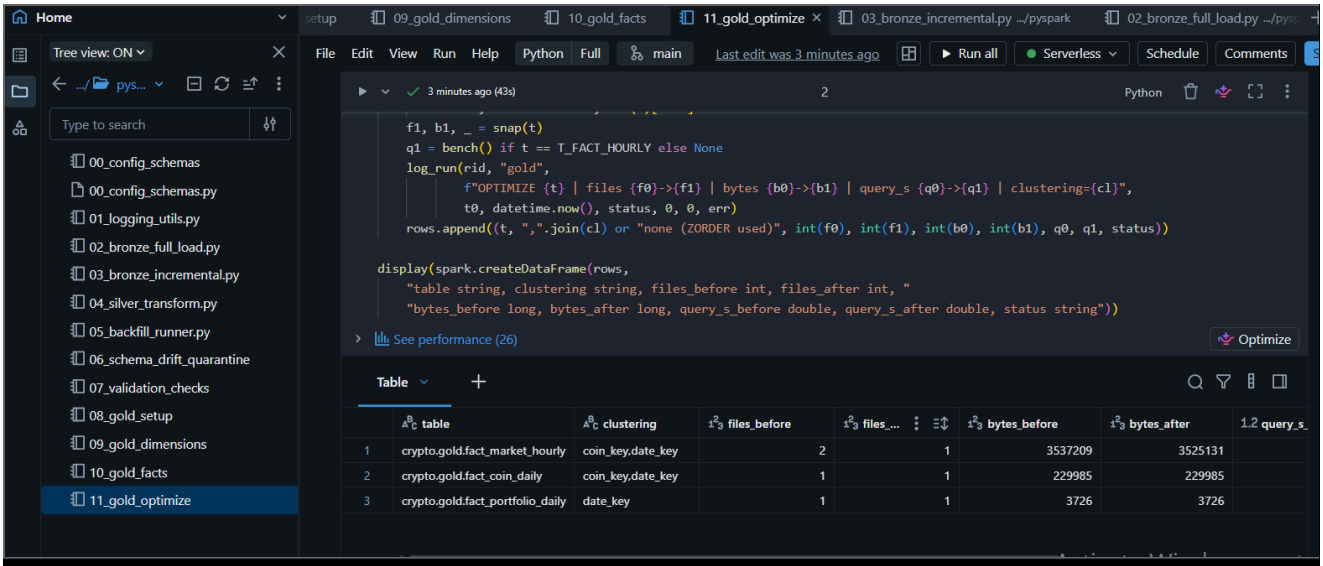


Figure 4. Output of notebook 11: file counts and sizes before and after OPTIMIZE.

Only the largest table had more than one file, so only it changed. At this data volume the benefit is small; the setup is what matters, because the same configuration keeps file counts low as the history grows.

4 Decision log

Partitioning and clustering

Decision: no partitioned columns. The fact tables use liquid clustering on the columns that dashboards filter on.

- The whole warehouse holds about 81,000 source observations and roughly 3.5 MB in its largest table. Partitioning by date or coin would create one tiny file per partition (about 31 dates, or 94 coins), which slows every query and wastes metadata work.
- The expected queries filter by coin and by date. Clustering on `coin_key` and `date_key` gives data skipping on exactly those columns without fixing a physical layout.
- Liquid clustering can be changed later with `ALTER TABLE ... CLUSTER BY`. A partition scheme cannot be changed without rewriting the table.
- At enterprise scale, with years of hourly data, the first change would be clustering Silver by `coin_id` and partitioning nothing finer than month.

Schema drift

Decision: quarantine, not automatic schema evolution.

- Every record is checked against the expected key set (`coin_id, timestamp, price, market_cap, total_volume`). A record with an extra key, a missing key, or a null `coin_id`, `timestamp` or `price` is written to `crypto.bronze.quarantine` with the reason (for example `schema drift: extra=['rank'], missing=[]`), the source file and the batch id. The rest of the batch loads normally.
- This was chosen over `mergeSchema` because an unexpected column or a changed type usually signals a problem upstream; adding it silently to Bronze would let it flow into Silver and Gold without review. Quarantine keeps the pipeline running and keeps the record for inspection and replay.
- Notebook 06 runs the same check as a separate step against any file, so a new source file can be inspected before it is loaded.

- Result on the real source files: no drift occurred. Both files conformed, 0 rows were quarantined, and both runs logged SUCCESS (notebook 06 output and run R007).

PII

Decision: no masking was required.

- The dataset is public market data: coin identifiers, prices, market capitalisation and traded volume. It contains no personal data, so no column is pseudonymised or removed.
- The Databricks personal access token that Power BI uses is the only credential in the project. It is stored in Power BI's credential store and is never committed to GitHub.
- In a production setting with sensitive data, access would be governed with Unity Catalog grants on the Gold schema and a service principal for the BI connection.

Other engineering decisions

Decision	Reason
Explicit StructType schemas, no inference	Stable types, no surprises when a file changes, required by the brief.
Business key (coin_id, ts_ms) with MERGE and update-only-if-changed	Unique per coin per hour. Re-runs show 0 inserted, 0 updated, which is the idempotency evidence.
Silver price rule: keep price >= 0 and non-null	Rows with null or negative price, or negative market cap or volume, are excluded from Silver. Bronze keeps every raw value, so nothing is lost.
Deterministic surrogate keys (xxhash64)	Identity columns change on re-run and would break an idempotent MERGE.
Two coin-level facts: hourly and daily	Hourly serves intraday questions (5 to 8). The daily aggregate serves performance and risk (1 to 4) without scanning hourly rows.
fact_portfolio_daily, dim_scenario and a bridge table	Question 11 needs scenario weights joined to daily returns by market condition.
Gold recomputed from Silver, then merged	Data is small, so this is simple and idempotent. At scale: watermark or Change Data Feed.
Orchestrator notebook	One run executes ingestion, Silver, Gold, validation and BI views, so Gold refreshes automatically after Silver and the whole run is logged.
Two Databricks workspaces, GitHub as the shared source	Free Edition allows one user per workspace. Power BI connects to one workspace.

5 Spark execution and DAG analysis

The most complex transformation is the `fact_coin_daily` aggregation in notebook 10. It reads Silver, applies window functions per coin (previous price, previous volume, volume mean and standard deviation), derives hourly measures and then aggregates to one row per coin and day. The plan was captured with `df.explain("formatted")`. Free Edition runs on serverless compute, which does not expose the classic Spark UI, so execution was analysed with the plan and the Query Profile statistics that Databricks shows under each notebook cell.

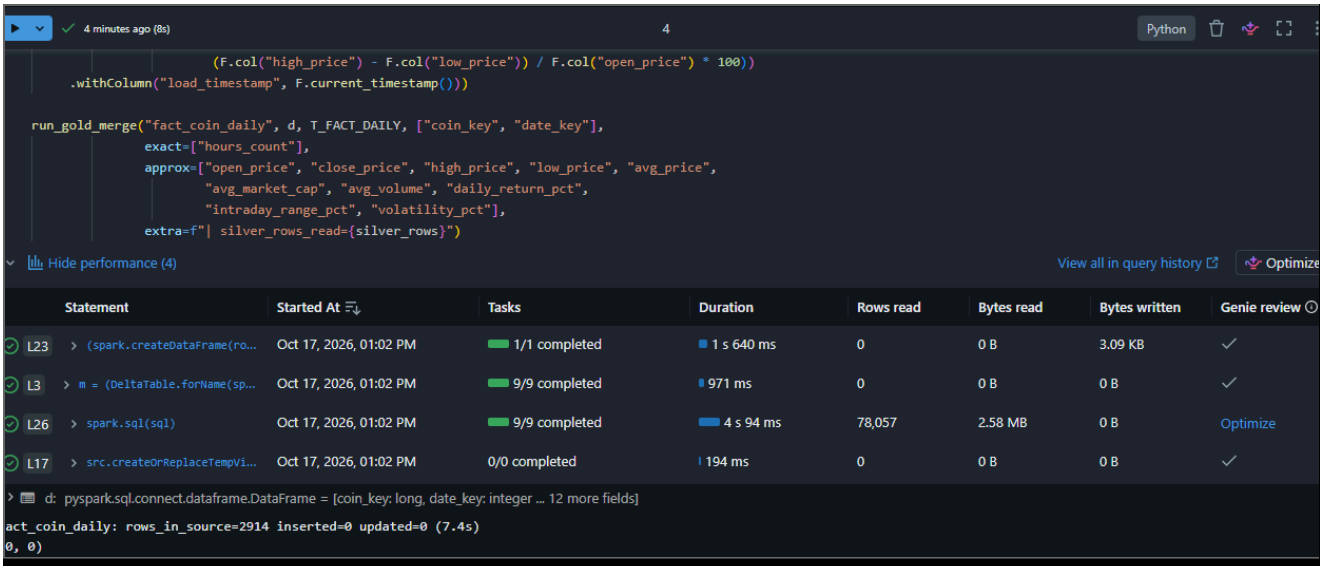


Figure 5. Execution statistics of the fact_coin_daily step. The MERGE statement (L26) completed 9 of 9 tasks, read 78,057 rows and 2.58 MB. The printed result shows `rows_in_source=2914 inserted=0 updated=0` on the idempotent re-run.

Physical plan, read from the bottom

Step	Operator	What Spark does
1	`PhotonScan parquet`	Reads the Silver Delta table.
2 to 4	`PhotonShuffleExchange`	**Shuffle 1.** Rows are redistributed by `coin_id` so that all hours of one coin sit in the same partition.
5	`PhotonSort`	Sorts each coin's rows by `event_ts`, required for `lag()`.
6	`PhotonWindow`	Previous price and volume (`lag`), plus mean and standard deviation of volume for the spike flag.
7	`PhotonProject`	Computes hourly return, volume change, volume-to-market-cap ratio, `coin_key` and `date_key`.
8	`PhotonGroupingAgg (partial)`	Map-side aggregation per (coin_key, date_key) before the second shuffle.
9 to 11	`PhotonShuffleExchange`	**Shuffle 2.** Partial aggregates for the same (coin_key, date_key) are brought together.
12	`PhotonGroupingAgg (final)`	Open and close price (`min_by`, `max_by` on `event_ts`), high, low, averages, volatility, hours_count.
13	`PhotonProject`	Daily return, intraday range, `load_timestamp`.
14, 15	`ColumnarToRow, ResultStage`	Returns the result to the MERGE.

How Spark executed it

- **Two shuffles.** Shuffle 1 is needed because window functions require each coin's rows together and ordered. Shuffle 2 is needed for the group-by on (coin_key, date_key). Shuffles move data across the network and write it to disk, which makes them the most expensive operations in the plan, and they mark the stage boundaries in the DAG.
- **Two-phase aggregation.** The partial aggregate (step 8) shrinks the data before shuffle 2; the final aggregate (step 12) combines the partial results. This is the standard way Spark limits shuffle volume for a group-by.

- **Photon.** Every operator runs in the vectorised Photon engine, so the plan has no row-based operators until the final conversion to rows for the MERGE.
- **Adaptive Query Execution.** AQE is enabled and chooses shuffle partition counts at run time. With about 66,000 rows most shuffle partitions are tiny, and AQE coalesces them rather than running hundreds of empty tasks.
- **Predicate pushdown.** This refresh reads the whole Silver table without a `WHERE` clause, so there is no filter to push down. Pushdown and file skipping apply to the dashboard queries, which filter on `coin_key` and `date_key`; liquid clustering on those columns lets Spark skip files whose statistics fall outside the filter.
- **Avoiding the first shuffle.** At this size its cost is negligible. At enterprise scale, clustering Silver by `coin_id` would reduce the data that shuffle 1 has to move.

6 Power BI dashboard design

Power BI connects directly to a Databricks SQL warehouse with the server hostname, HTTP path and a personal access token, and reads the Gold tables and the five BI views created by notebook 14 (`vw_market_overview`, `vw_coin_daily_performance`, `vw_portfolio_scenarios`, `vw_scenario_composition`, `vw_intraday_analysis`). The monitoring page reads `pipeline_execution_logs` and `quarantine` directly. Relationships follow the star schema: one-to-many, single direction, from each dimension to its fact table.

Relationship	Many side
`dim_coin[coin_key]`	`fact_market_hourly, fact_coin_daily, bridge_scenario_coin`
`dim_date[date_key]`	`fact_market_hourly, fact_coin_daily, fact_portfolio_daily`
`dim_hour[hour_key]`	`fact_market_hourly`
`dim_scenario[scenario_key]`	`bridge_scenario_coin, fact_portfolio_daily`

Pages and visuals

Page	Visuals	Questions
Market Overview	KPI cards (coins tracked, latest total market cap, average daily return); line chart of close price indexed to 100 by coin; bar chart of average market cap versus average volume; scatter of volume-to-market-cap ratio against market cap.	1, 2
Risk and Performance	Bar chart of volatility by coin (top 15); column chart of average daily return by day of week; matrix heatmap of daily return by coin and date.	1, 3, 4
Intraday Analysis	Average hourly return and volume by hour of day; scatter of volume change against hourly return with spikes highlighted; last-24-hour price and volume for a selected coin; last 24 hours versus historical average by hour.	5, 6, 7, 8
Decision Support	Cumulative return by allocation scenario; matrix of scenario by market condition (Bull, Bear, Sideways).	11
Pipeline Monitoring	Runs by layer and status; rows inserted and updated and duration per run; quarantine reasons; table of failed runs with error messages.	Audit and error logs

The dashboard exceeds the minimum of three distinct visuals: it combines time-series trends, categorical breakdowns and KPI cards, and the Pipeline Monitoring page adds audit and error-log visuals fed from `pipeline_execution_logs` and `quarantine`. The delivered `.pbix` file contains the final pages.

7 Validation and data quality

Check	Where	Result
Bronze rows equal Silver rows	Notebook 07	66,401 = 66,401
Duplicate keys in Bronze and Silver	Notebook 07	0
Invalid values in Silver (null, negative)	Notebook 07	0
Rows in quarantine	Notebook 07	0
Bronze re-run of the full file	Run R003	0 inserted, 0 updated
Incremental file over existing data	Run R002	14,474 read, 0 inserted, 0 updated
Silver re-run	Run R005	0 inserted, 0 updated
Gold fact_coin_daily re-run	Notebook 10	2,914 rows in source, 0 inserted, 0 updated
Bronze and Silver not empty; no duplicate Silver keys; required fields and valid prices; scenario weights sum to 1; portfolio returns populated	Notebook 13, runs in every orchestrator pass	PASS or FAIL per check, logged to `pipeline_execution_logs` (layer `validation`)

8 Cost and FinOps

Databricks Free Edition has no billing data, so no credits were consumed and cost cannot be read from an invoice. The figures below are estimates built from measured run times and stated assumptions. The method is: **cost per run = duration (hours) x DBU per hour x price per DBU**.

Measured run times

Step (notebook 09 and 10)	Duration (s)	Rows aggregated
`fact_market_hourly merge`	7.2	66,401
`fact_coin_daily merge`	6.3	2,914
`fact_portfolio_daily merge`	6.3	155
`bridge_scenario_coin merge`	9.3	282
`dim_scenario merge`	6.3	5
`fact_coin_daily re-run (0 changes)`	7.4	2,914

Durations are taken from `pipeline_execution_logs` and the notebook output. The five Gold merge steps together take about 35 seconds. Most of each step is fixed overhead (query planning, MERGE and Delta log commit), not data volume.

Assumptions

- Full orchestrated run (ingest, Silver, Gold, OPTIMIZE, validation, BI views): **5 minutes**. This is an estimate from the measured steps plus startup and the remaining notebooks, not a logged value.
- Compute size: **2 DBU per hour** (small job compute).
- Price per DBU: **USD 0.15** (jobs compute, low case; a published list rate for the AWS Premium plan) and **USD 0.40** (high case, an allowance for serverless or a higher tier). List prices change; they must be checked against the current Databricks pricing page.

- Cloud virtual-machine cost for classic compute and storage are excluded. Storage is negligible: the whole Gold layer is under 4 MB.

Estimated cost

Scenario	Per run (USD)	Per year (USD)
Typical daily incremental run, 5 min, 1 run per day	0.025 to 0.067	9.1 to 24.3
Enterprise: same pipeline hourly for a year (8,760 runs), 5 min each	0.025 to 0.067	**219 to 584**
Enterprise, optimised: hourly, 1 min per run (incremental only)	0.005 to 0.013	**44 to 117**

The pipeline is small, so the cost is dominated by how long compute is switched on, not by the data. Running hourly for a year costs about 24 times the daily schedule, and almost all of that is startup and fixed overhead.

Optimisation

- **Job compute with auto-termination:** pay only while a run is active; do not use an always-on all-purpose cluster.
- **Process only new data:** replace the full Gold recompute with a watermark on `load_timestamp` or Delta Change Data Feed, so each run touches only the new hour.
- **Run OPTIMIZE and the full validation daily, not hourly:** this removes the longest steps from 23 of every 24 runs and is the main reason the optimised run drops to about one minute.
- **Event-driven triggers:** start a run when a file arrives (Auto Loader file arrival) instead of a fixed schedule when the source is irregular.
- **Right-size compute and review Photon:** keep the cluster at the smallest size that meets the hourly deadline; Photon carries a DBU premium that only pays off on larger scans.
- **Use a serverless SQL warehouse for Power BI with auto-stop,** and refresh the dashboard on a schedule rather than with live queries for every viewer.

9 Limitations and future work

- **Short history.** The data covers about 31 days. Day-of-week results rest on about four observations per weekday, and the Bull, Bear and Sideways classification reflects a single month, so findings are indicative, not statistically strong.
- **Overlapping incremental file.** The incremental file lies inside the full-load window and inserted nothing. It proves idempotency, but a truly new window would be needed to show the incremental path adding rows.
- **Prediction (questions 9 and 10)** was optional and is not implemented. A direction classifier would add a `fact_prediction` table to the model.
- **FinOps figures are estimates,** because Free Edition provides no billing data. They should be replaced with logged orchestrator durations and real DBU consumption when the pipeline runs on a paid workspace.
- **Future work:** foreign-key orphan and Silver-to-Gold reconciliation checks in notebook 13, streaming or Auto Loader ingestion, Unity Catalog access controls and a service principal for Power BI, a scheduled Databricks Job in place of the manually triggered orchestrator, and automated data-quality alerts on failed runs.